

threads

process

program



What We'll Cover

what threads, processes, and programs are,
and how they relate to each other

how parallelism and concurrency
actually work under the hood

real-world design choices: when
to use threads vs processes



Dedicated Resources



Shared Resources



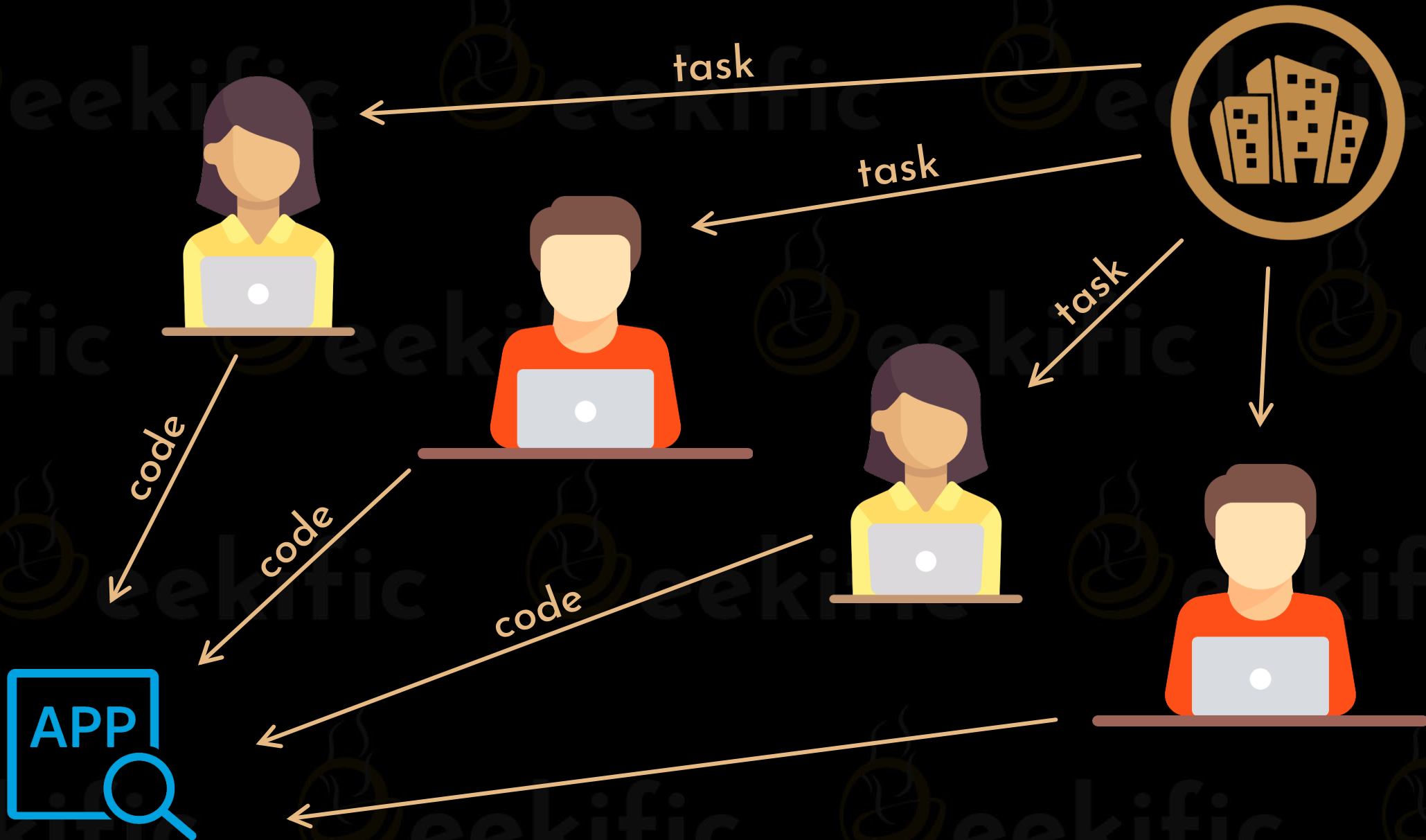
The Company Analogy

each employee has personal tools: a laptop, a notebook, a coffee mug

the company also has shared resources: a central server, a monitoring screen, the cafeteria

employees use both types of resources to complete their assigned tasks





Combined Efforts

some employees build entire applications,
others develop just a single feature

their combined work produces the final product

each employee is the smallest unit
contributing to the company's goal





an employee to our company
is like a **thread** to a **process**

smallest unit



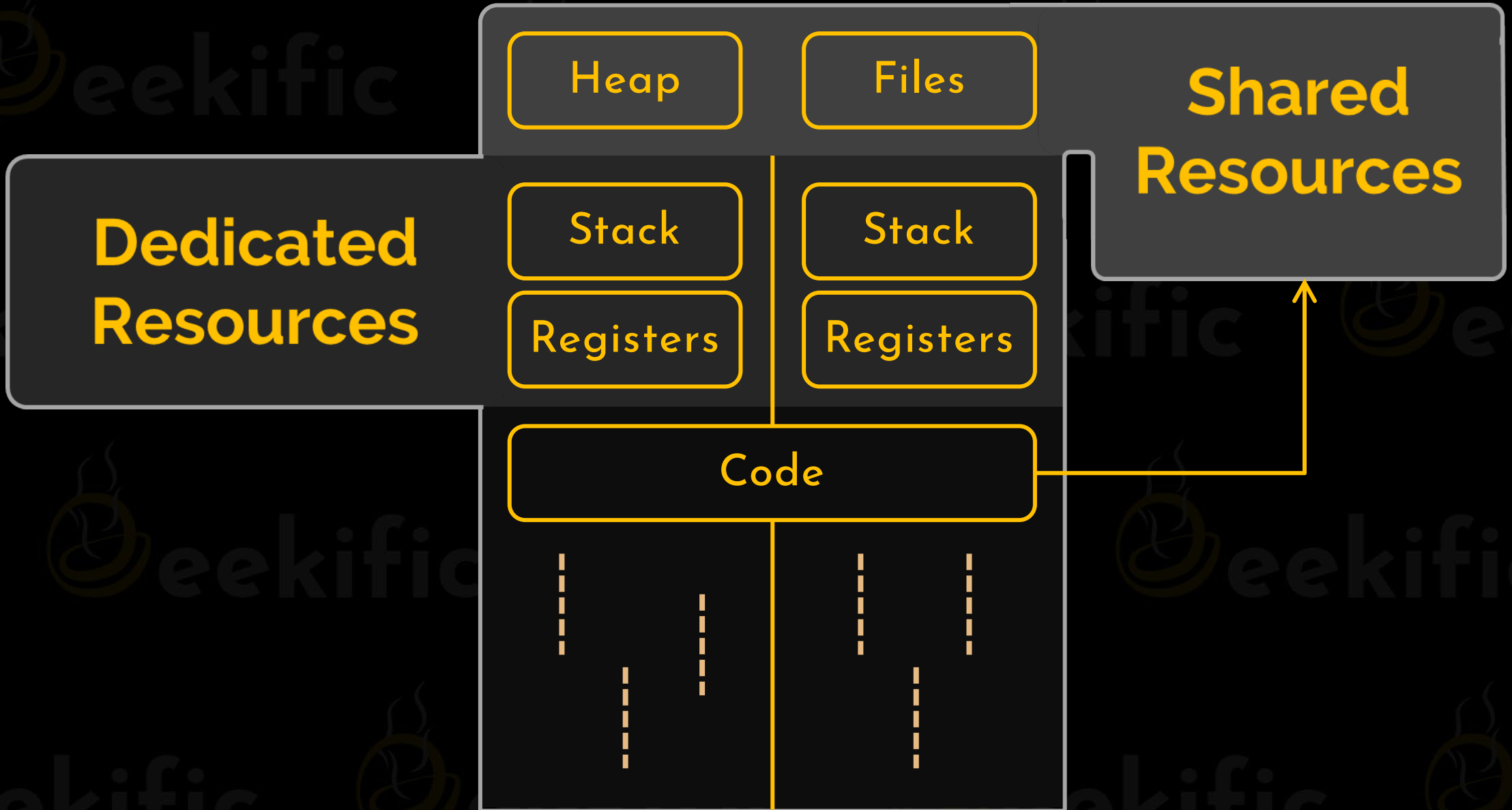
single-threaded process

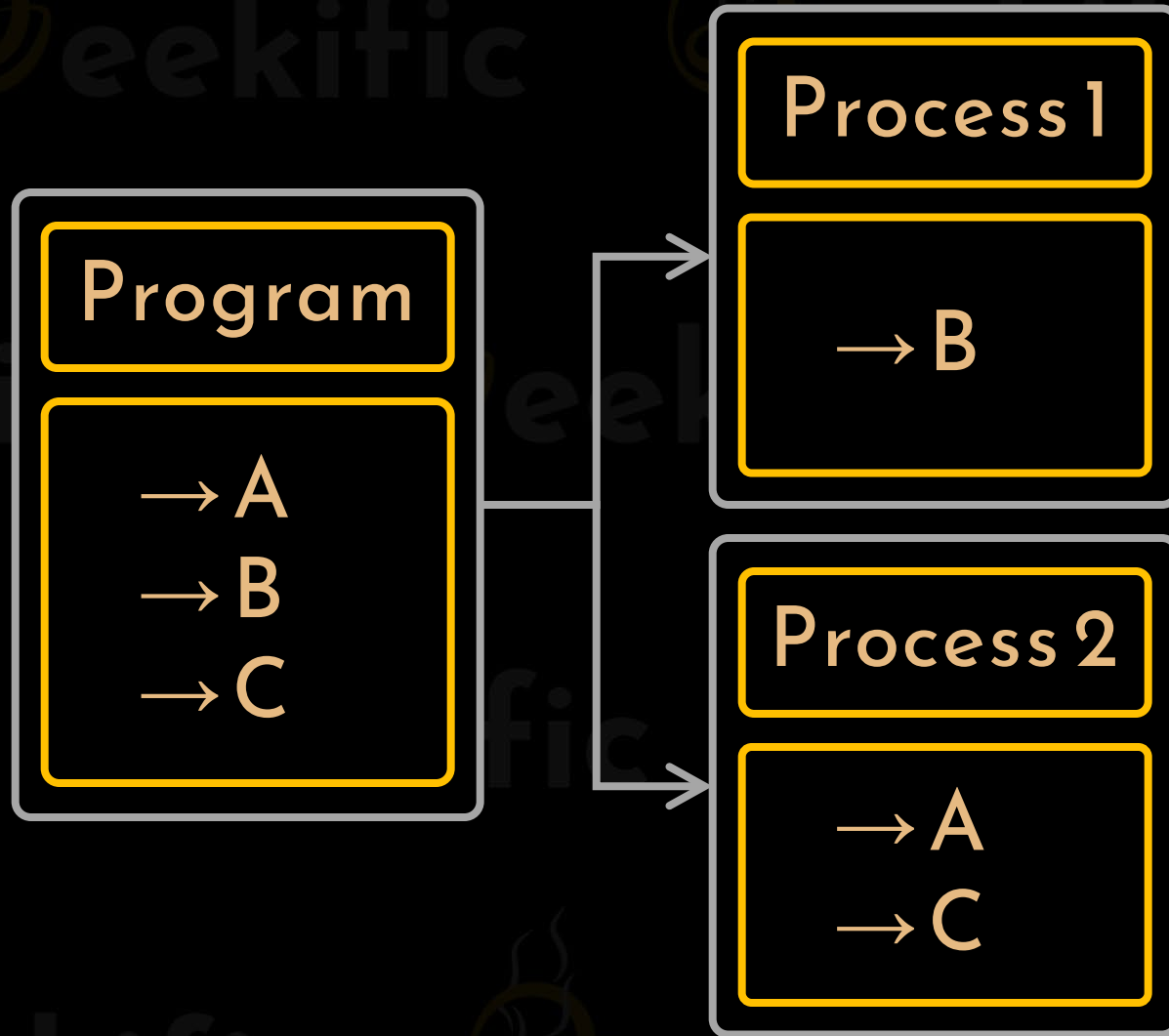
one thread executes all the
sequence of operations

multi-threaded process

the joint effort of all threads will result in the
execution of the sequence of operations







the program is
the blueprint; the
process is the live
execution of it

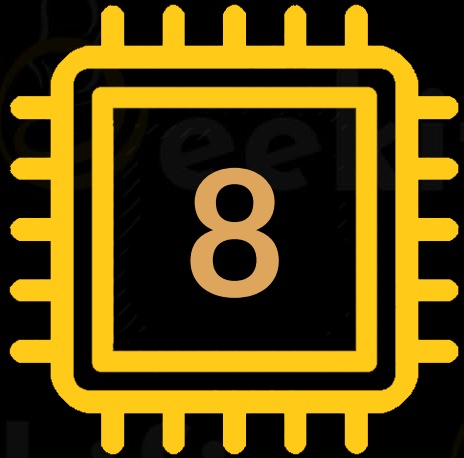
like a class vs an
object, one program
can spawn several
processes





can run 4 threads
in parallel

each operation is
allocated tiny time slots



can run 8 threads
in parallel



The Illusion of Parallelism

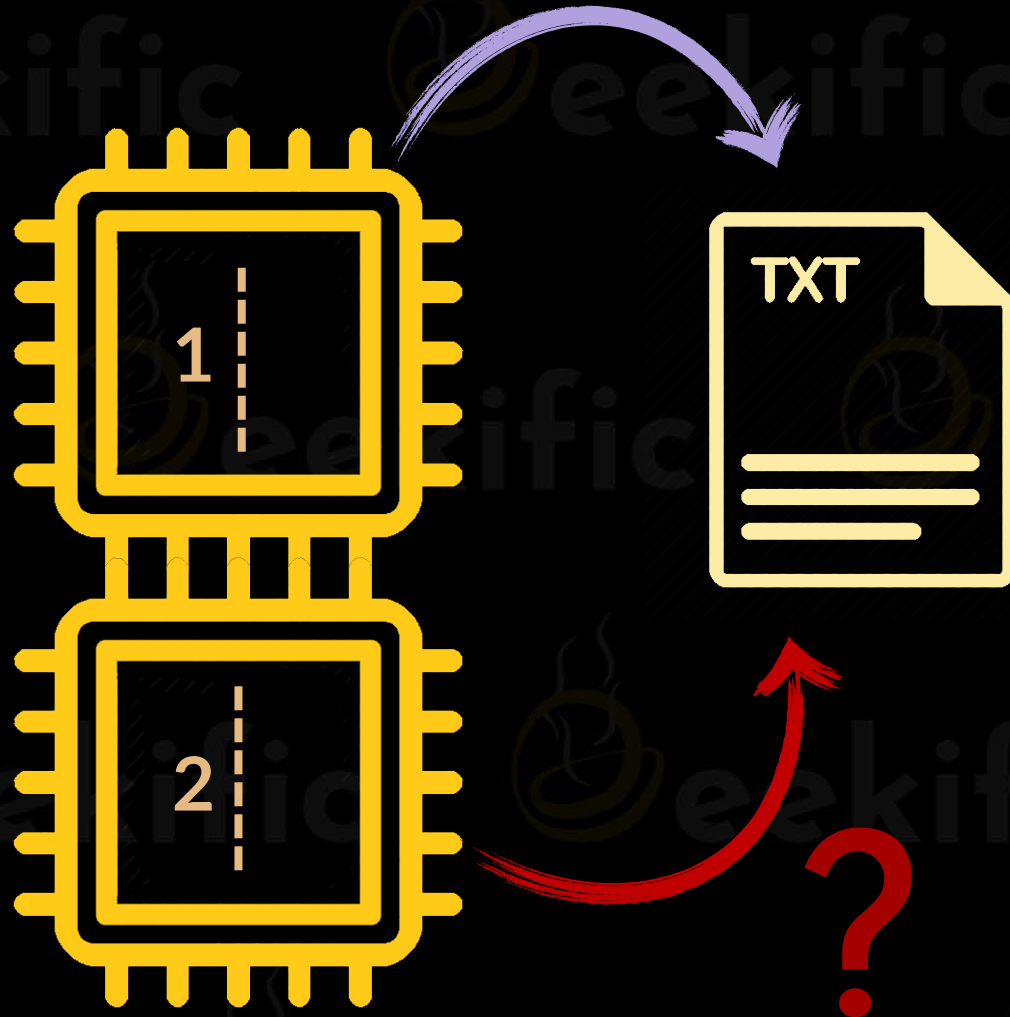
a CPU with 8 cores can theoretically run 8 threads simultaneously

in practice, we often run far more threads than available cores

the processor juggles threads in tiny time slots, less than milliseconds

this creates the illusion that all threads run together





here, thread-2
will have to wait
for thread-1

from parallel
to concurrent
threads



Parallelism vs Concurrency

parallelism is about hardware: multiple cores
running threads at the same instant

when two parallel threads need the same
resource, one must wait for the other

concurrency is about software: managing
shared access so threads take turns



Scheduler

part of the operating system

assigns CPU time to each thread

prioritizes tasks

creates the illusion of parallelism
via tiny time slots



scheduler
process
program



What is a Thread?

the smallest unit of execution managed by the scheduler – in most languages, you create one by giving it a function to run

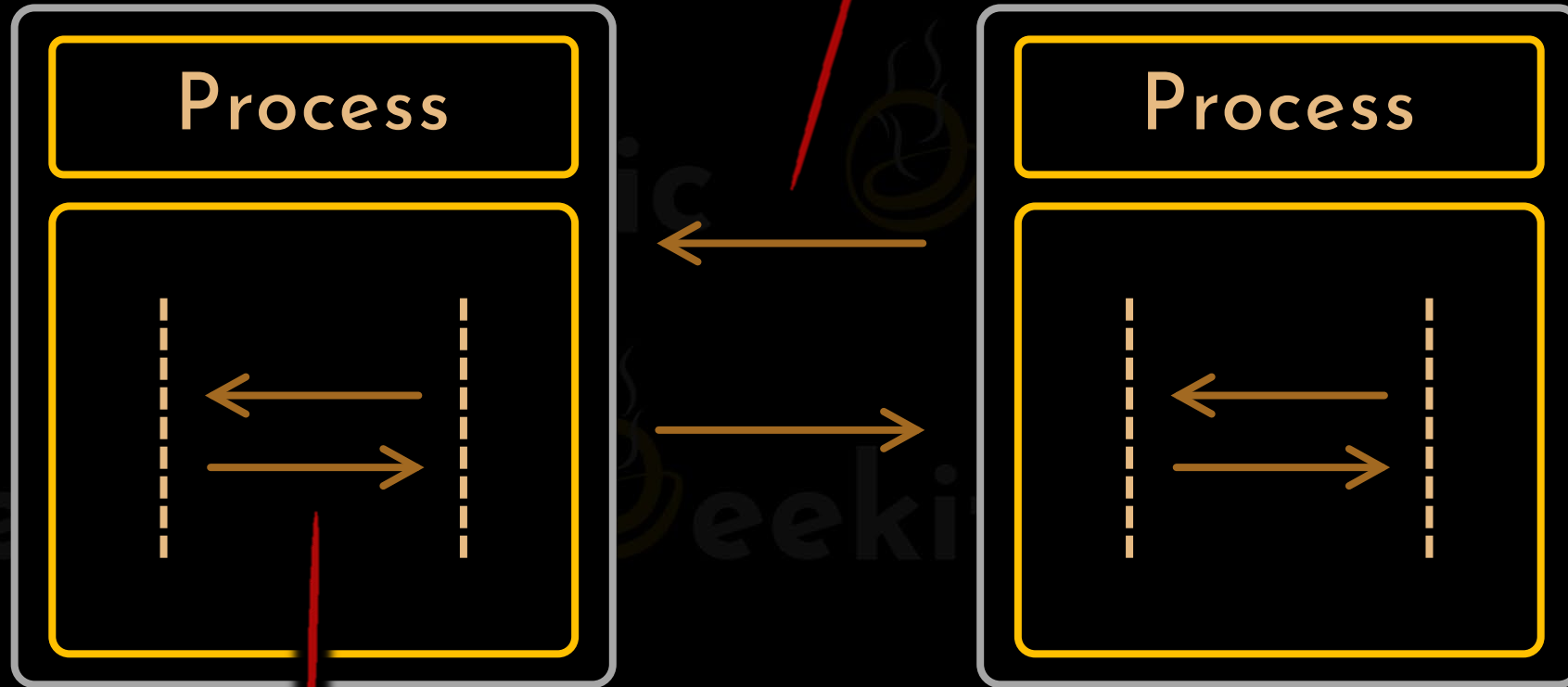
multiple threads can exist within a process and share its resources

as long as one thread is running, the process is alive; remove all threads, and it ceases to exist



is using threads
always better?

inter-process communication



inter-thread communication



Processes vs Threads

threads share memory, making communication fast,
but a crash in one can kill the entire process

processes are isolated: a crash in one won't affect others,
but communication between them is slower

the choice is a tradeoff between speed of
communication and safety of isolation



Name	57% CPU	41% Memory	1% Disk	1% Network	6% GPU	GPU engine
Apps (6)						
Blizzard Battle.net App (32 bit) (3)	0%	113.6 MB	0 MB/s	0 Mbps	0%	
Google Chrome (33)	40.7%	1,537.2 MB	1.0 MB/s	0 Mbps	3.0%	GPU 1 - 3D
Google Chrome	4.0%	71.8 MB	0 MB/s	0 Mbps	3.0%	GPU 1 - 3D
Google Chrome	1.2%	78.3 MB	0 MB/s	0 Mbps	0%	
Google Chrome	3.6%	36.5 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	6.3 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	2.0%	62.5 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	93.5 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	32.1 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	44.9 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	86.8 MB	0.2 MB/s	0 Mbps	0%	
Google Chrome	1.5%	88.7 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	11.6 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	29.7 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	11.2 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0.1%	12.1 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	4.0%	71.4 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	25.6 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	4.1 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0.2%	47.1 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	22.8 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	11.1 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	5.7 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	25.4 MB	0 MB/s	0 Mbps	0%	

each one of
these is a
process and
not a thread

Resource Monitor					
File Monitor Help					
Overview CPU Memory Disk Network					
CPU 21% CPU Usage 123% Maximum Frequency					
☐ Image	PID	Description	Status	Threads	CF
☐ chrome.exe	17952	Google Chrome	Runni...	20	
☐ chrome.exe	3408	Google Chrome	Runni...	15	
☐ chrome.exe	14216	Google Chrome	Runni...	8	
☐ chrome.exe	18152	Google Chrome	Runni...	15	
☐ chrome.exe	4900	Google Chrome	Runni...	16	
☐ chrome.exe	16800	Google Chrome	Runni...	17	
☐ chrome.exe	9592	Google Chrome	Runni...	16	
☐ chrome.exe	18388	Google Chrome	Runni...	15	
☐ chrome.exe	16508	Google Chrome	Termin...	19	
☐ chrome.exe	9624	Google Chrome	Runni...	8	
☐ chrome.exe	15288	Google Chrome	Runni...	17	
☐ chrome.exe	8764	Google Chrome	Runni...	15	
☐ chrome.exe	1816	Google Chrome	Runni...	15	
☐ chrome.exe	15740	Google Chrome	Termin...	14	
☐ chrome.exe	15264	Google Chrome	Runni...	14	
☐ chrome.exe	10628	Google Chrome	Runni...	17	
☐ chrome.exe	9232	Google Chrome	Runni...	14	
☐ chrome.exe	14396	Google Chrome	Runni...	14	
☐ chrome.exe	18824	Google Chrome	Runni...	14	
☐ chrome.exe	10212	Google Chrome	Runni...	14	
☐ chrome.exe	10332	Google Chrome	Runni...	14	



Why Multiple Processes?

each tab connects to servers, downloads media,
and renders content independently

tabs can be detached into separate windows with new responsibilities

if one tab crashes, it doesn't take down the whole browser

inactive windows get less priority, freeing resources for active ones



Name	57% CPU	41% Memory	1% Disk	1% Network	6% GPU	GPU engine
Apps (6)						
Blizzard Battle.net App (32 bit) (3)	0%	113.6 MB	0 MB/s	0 Mbps	0%	
Google Chrome (33)	40.7%	1,537.2 MB	1.0 MB/s	0 Mbps	0%	GPU 1
Google Chrome	4.0%	71.8 MB	0 MB/s	0 Mbps	0%	
Google Chrome	1.2%	78.3 MB	0 MB/s	0 Mbps	0%	
Google Chrome	3.6%	36.5 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	6.3 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	2.0%	62.5 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	93.5 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	32.1 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	44.9 MB	0 MB/s	0 Mbps	0%	
Google Chrome	1.5%	88.1 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	6.8 MB	0 MB/s	0 Mbps	0%	
Google Chrome	11.6%	110.4 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	30.7 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	1.2 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0.1%	37.2 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	4.0%	71.4 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	25.6 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0.2%	17.1 MB	0.1 MB/s	0 Mbps	0%	
Google Chrome	0%	22.8 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	33.9 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0.1%	11.1 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	5.7 MB	0 MB/s	0 Mbps	0%	
Google Chrome	0%	25.4 MB	0 MB/s	0 Mbps	0%	

less glitches

better memory management

higher performance

Resource Monitor					
File Monitor Help					
Overview CPU Memory Disk Network					
CPU 21% CPU Usage 123% Maximum Frequency					
☐	Image	PID	Description	Status	Threads
☐	chrome.exe	17952	Google Chrome	Runni...	20
☐	chrome.exe	3408	Google Chrome	Runni...	15
☐	chrome.exe	14216	Google Chrome	Runni...	8
☐	chrome.exe	18152	Google Chrome	Runni...	15
☐	chrome.exe	4900	Google Chrome	Runni...	16
☐	chrome.exe	16800	Google Chrome	Runni...	17
☐	chrome.exe	9592	Google Chrome	Runni...	16
☐	chrome.exe	18388	Google Chrome	Runni...	15
☐	chrome.exe	16508	Google Chrome	Termin...	19
☐	chrome.exe	9624	Google Chrome	Runni...	8
☐	chrome.exe	15288	Google Chrome	Runni...	17
☐	chrome.exe	8764	Google Chrome	Runni...	15
☐	chrome.exe	1816	Google Chrome	Runni...	15
☐	chrome.exe	15740	Google Chrome	Termin...	14
☐	chrome.exe	15264	Google Chrome	Runni...	14
☐	chrome.exe	10628	Google Chrome	Runni...	17
☐	chrome.exe	9232	Google Chrome	Runni...	14
☐	chrome.exe	14396	Google Chrome	Runni...	14
☐	chrome.exe	18824	Google Chrome	Runni...	14
☐	chrome.exe	10212	Google Chrome	Runni...	14
☐	chrome.exe	10332	Google Chrome	Runni...	14



Putting It All Together

a program is the blueprint that defines what can be done

a process is the live execution of that program

threads are the workers inside a process, carrying out the actual operations

the scheduler orchestrates it all, giving each thread its turn on the CPU

